```
!pip install -q yt-dlp pydub librosa soundfile sarvamai datasets huggingface_hub pandas
!apt-get install -qq ffmpeg
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
path   = "/content/drive/MyDrive/sarvam_assignment"
dataset = f"{path}/dataset/ind_eng"
output = f"{path}/eng_segments"
```

```
PANEL_VIDEOS = {
    "en_v4": ("https://youtu.be/x3TvDatWtqE", 0),
}

min_clip_duration = 5
max_clip_duration = 28
min_snr_db        = 15
trim_end          = 5

import os
os.makedirs(dataset, exist_ok=True)
os.makedirs(output,  exist_ok=True)
```

```
from google.colab import userdata

def _get_secret(name):
    try:
        value = userdata.get(name)
    except Exception:
        value = ""
    if not value:
        raise ValueError(
            f"Secret '{name}' not found or notebook access not granted. "
        )
    return value

SARVAM_API_KEY  = _get_secret("SARVAM_API_KEY1")
```

```
import librosa
import numpy as np
import soundfile as sf
import json
import time
import math
import subprocess
import random
import requests
from datetime import datetime
from collections import Counter
from pathlib import Path
import IPython.display as ipd
import pandas as pd

from sarvamai import SarvamAI
sarvam_client = SarvamAI(api_subscription_key=SARVAM_API_KEY)
```

```
# Download and load video
def download_video(url, video_id):
    path1 = str(f"{dataset}/{video_id}.wav")
    if os.path.exists(path1):
        print(f"  Already downloaded: {video_id} — skipping.")
        return path1

    print(f"  Downloading {video_id}...")
    out_template = str(path1.replace(".wav", ".%(ext)s"))

    result = subprocess.run([
        "yt-dlp", "-x",
```

```
            "--audio-format", "wav",
            "--audio-quality", "0",
            "-o", out_template,
            str(url)
        ], capture_output=True, text=True)

        if result.returncode != 0:
            print(f"  ERROR: {result.stderr[:300]}")
            return None

        print(f"  Saved: {path1}")
        return path1

    def load_video(path, sr=16000, start_sec=0, end_sec=0):
        audio, _ = librosa.load(path, sr=sr, mono=True)

        start = int(start_sec * sr)
        end = int(len(audio) - (end_sec * sr))
        start = max(0, start)
        end = max(start + 1, end)
        trimmed = audio[start:end]
        duration_mins = (len(trimmed) / sr) / 60
        print(f"Trimmed {start_sec}s intro and {end_sec}s outro, {duration_mins:.1f} min remaining")
        return trimmed, sr
```

```
    def compute_snr(segment):
        rms         = np.sqrt(np.mean(segment ** 2))
        noise_floor = np.percentile(np.abs(segment), 10)
        return 20 * np.log10(rms / (noise_floor + 1e-8))

    def add_fades(segment, sr, fade_ms=50):
        n   = int(fade_ms * sr / 1000)
        seg = segment.copy()
        seg[:n]  *= np.linspace(0, 1, n)
        seg[-n:] *= np.linspace(1, 0, n)
        return seg
```

```
    def split(duration, max_dur):
        if duration <= max_dur:
            return [(0, duration)]
        n     = math.ceil(duration / max_dur)
        piece = duration / n
        return [(i * piece, (i + 1) * piece) for i in range(n)]


    def transcribe(audio_path, language_code):

        with open(audio_path, "rb") as f:
            resp = requests.post(
                "https://api.sarvam.ai/speech-to-text",
                headers={"api-subscription-key": SARVAM_API_KEY},
                files={"file": (os.path.basename(audio_path), f, "audio/wav")},
                data={
                    "model":            "saaras:v3",
                    "mode":             "transcribe",
                    "language_code":    language_code,
                    "with_timestamps":  False,
                    "with_disfluencies": False,
                }
            )
        if resp.status_code == 200:
            return resp.json().get("transcript", "").strip()
        print(f"    REST ASR error {resp.status_code}: {resp.text[:200]}")
        return None
```

```
    import time

    def run_diarized_batch(file_paths, language_code,
                           out_dir="/tmp/sarvam_batch_out", timeout=1800, retries=3):
        for attempt in range(1, retries + 1):
            try:
                job = sarvam_client.speech_to_text_job.create_job(
                    model="saaras:v3",
                    mode="transcribe",
                    language_code=language_code,
```

```
                with_diarization=True,
            )
            job.upload_files(file_paths=file_paths)
            job.start()
            job.wait_until_complete(timeout=timeout)

            results = job.get_file_results()
            for f in results["failed"]:
                print(f"    Batch ASR failed: {f['file_name']} -> {f['error_message']}")

            os.makedirs(out_dir, exist_ok=True)
            job.download_outputs(output_dir=out_dir)

            parsed = {}
            for fp in file_paths:
                json_path = os.path.join(out_dir, f"{os.path.basename(fp)}.json")
                if not os.path.exists(json_path):
                    continue
                with open(json_path, "r", encoding="utf-8") as fh:
                    data = json.load(fh)
                entries     = (data.get("diarized_transcript") or {}).get("entries", [])
                speaker_ids = sorted(set(e["speaker_id"] for e in entries))
                parsed[fp]  = {
                    "transcript":  (data.get("transcript") or "").strip(),
                    "speaker_ids": speaker_ids,
                }
            return parsed

        except Exception as e:
            print(f"    Attempt {attempt}/{retries} failed: {e}")
            if attempt < retries:
                print(f"    Retrying in 30s...")
                time.sleep(30)
            else:
                print(f"    All {retries} attempts failed for this batch.")
                return {}
```

```
def verify_single_speaker(accepted):

    clean, rejected = [], []
    for b in range(0, len(accepted), 20):
        chunk   = accepted[b:b + 20]
        results = run_diarized_batch(
            [s["file"] for s in chunk],
            chunk[0]["lang_code"]
        )
        for seg in chunk:
            r = results.get(seg["file"])
            if r and len(r["speaker_ids"]) > 1:
                rejected.append({
                    "id":       seg["id"],
                    "duration": seg["duration_sec"],
                    "reason":   f"overlap_contamination:{len(r['speaker_ids'])}_voices",
                })
                try:
                    os.remove(seg["file"])
                except OSError:
                    pass
                print(f"    {seg['id']}: Rejected (overlap contamination)")
            else:
                clean.append(seg)
    return clean, rejected
```

```
def process_interview_video(video_id, url, output, lang_code,
                            start_sec, end_sec, num_speakers=None,
                            diarization_timeout=1800, verify_clips=True):
    print(f"  {video_id} | {lang_code} | num_speakers={num_speakers}")

    # Step 1: download
    raw_path = download_video(url, video_id)
    if raw_path is None:
        return [], [{"id": video_id, "reason": "download_failed", "duration": 0}]

    # Step 2: load + trim
    audio, sr    = load_video(raw_path, start_sec=start_sec, end_sec=end_sec)
    trimmed_path = f"{dataset}/{video_id}_trimmed.wav"
```

```
        sf.write(trimmed_path, audio, sr)

        # Step 3: diarize the FULL trimmed file in one shot
        print(f"  Submitting full audio for diarization...")
        job = sarvam_client.speech_to_text_job.create_job(
            model="saaras:v3",
            mode="transcribe",
            language_code=lang_code,
            with_diarization=True,
            num_speakers=num_speakers,   # 5 for this video; None = auto-detect
        )
        job.upload_files(file_paths=[trimmed_path])
        job.start()
        job.wait_until_complete(timeout=diarization_timeout)

        results = job.get_file_results()
        if not results["successful"]:
            err = results["failed"][0]["error_message"] if results["failed"] else "unknown"
            return [], [{"id": video_id, "reason": f"diarization_failed:{err}", "duration": 0}]

        out_dir = "/tmp/sarvam_batch_out"
        os.makedirs(out_dir, exist_ok=True)
        job.download_outputs(output_dir=out_dir)

        json_path = os.path.join(out_dir, f"{os.path.basename(trimmed_path)}.json")
        with open(json_path, "r", encoding="utf-8") as f:
            data = json.load(f)
        turns = (data.get("diarized_transcript") or {}).get("entries", [])
        print(f"  Diarization returned {len(turns)} speaker turns")

        if not turns:
            return [], [{"id": video_id, "reason": "no_diarization_turns", "duration": 0}]

        # Step 4: slice each speaker turn into clips
        accepted, rejected = [], []
        seg_idx = 0

        for turn in turns:
            text     = (turn.get("transcript") or "").strip()
            spk      = turn["speaker_id"]
            t0       = turn["start_time_seconds"]
            duration = turn["end_time_seconds"] - t0
            pieces   = split(duration, max_clip_duration)

            for rel_s, rel_e in pieces:
                s, e      = t0 + rel_s, t0 + rel_e
                piece_dur = e - s
                seg_id    = f"{video_id}_seg{seg_idx:03d}"
                seg_idx  += 1

                # Duration gate
                if piece_dur < min_clip_duration:
                    rejected.append({"id": seg_id, "duration": round(piece_dur, 2),
                                     "reason": "too_short"})
                    continue

                # Extract audio chunk
                start_i = int(s * sr)
                end_i   = int(e * sr)
                chunk   = audio[start_i:end_i]

                # SNR gate
                snr = compute_snr(chunk)
                if snr < min_snr_db:
                    rejected.append({"id": seg_id, "duration": round(piece_dur, 2),
                                     "reason": f"low_snr:{snr:.1f}dB"})
                    continue

                # Text for this piece
                if len(pieces) == 1:
                    # Turn fits in one clip: use the diarization text directly
                    piece_text = text
                else:
                    # Turn was split: re-transcribe this piece to get accurate text
                    tmp_path = os.path.join(output, f"_tmp_{seg_id}.wav")
                    sf.write(tmp_path, add_fades(chunk, sr), sr)
                    piece_text = transcribe(tmp_path, lang_code)
```

```
                os.remove(tmp_path)

            if not piece_text or len(piece_text) <= 3:
                rejected.append({"id": seg_id, "duration": round(piece_dur, 2),
                                 "reason": "empty_transcript"})
                continue

            # Write clip
            chunk    = add_fades(chunk, sr)
            filepath = os.path.join(output, f"{seg_id}.wav")
            sf.write(filepath, chunk, sr)
            accepted.append({
                "id":           seg_id,
                "file":         filepath,
                "lang_code":    lang_code,
                "duration_sec": round(piece_dur, 2),
                "snr_db":       round(snr, 2),
                "source_url":   url,
                "transcript":   piece_text,
                "speaker_id":   f"{video_id}_spk{spk}",
            })

    # Step 5: contamination check
    if verify_clips and accepted:
        print(f"  Running contamination check on {len(accepted)} clips...")
        accepted, overlap_rejected = verify_single_speaker(accepted)
        rejected.extend(overlap_rejected)

    print(f"  Turns: {len(turns)} | Accepted: {len(accepted)} | Rejected: {len(rejected)}")
    return accepted, rejected
```

```
all_accepted = []
all_rejected = []

for video_id, url_data in PANEL_VIDEOS.items():
    url, start_sec = url_data
    acc, rej = process_interview_video(
        video_id    = video_id,
        url         = url,
        output      = output,
        lang_code   = "en-IN",
        start_sec   = start_sec,
        end_sec     = trim_end,
        num_speakers= 5,
        verify_clips= True,
    )
    all_accepted.extend(acc)
    all_rejected.extend(rej)

total_min = sum(s["duration_sec"] for s in all_accepted) / 60
print(f"\nTotal: {len(all_accepted)} segments | {total_min:.1f} min")
```

```
  en_v4 | en-IN | num_speakers=5
  Downloading en_v4...
  Saved: /content/drive/MyDrive/sarvam_assignment/dataset/ind_eng/en_v4.wav
Trimmed 0s intro and 5s outro, 29.0 min remaining
  Submitting full audio for diarization...
  Diarization returned 101 speaker turns
  Running contamination check on 113 clips...
    en_v4_seg016: Rejected (overlap contamination)
    en_v4_seg017: Rejected (overlap contamination)
    en_v4_seg030: Rejected (overlap contamination)
    en_v4_seg033: Rejected (overlap contamination)
    en_v4_seg034: Rejected (overlap contamination)
    en_v4_seg041: Rejected (overlap contamination)
    en_v4_seg043: Rejected (overlap contamination)
    en_v4_seg044: Rejected (overlap contamination)
    en_v4_seg045: Rejected (overlap contamination)
    en_v4_seg046: Rejected (overlap contamination)
    en_v4_seg048: Rejected (overlap contamination)
    en_v4_seg054: Rejected (overlap contamination)
    en_v4_seg058: Rejected (overlap contamination)
    en_v4_seg063: Rejected (overlap contamination)
    en_v4_seg076: Rejected (overlap contamination)
    en_v4_seg083: Rejected (overlap contamination)
    en_v4_seg084: Rejected (overlap contamination)
    en_v4_seg091: Rejected (overlap contamination)
    en_v4_seg098: Rejected (overlap contamination)
    en_v4_seg099: Rejected (overlap contamination)
```

```
    en_v4_seg102: Rejected (overlap contamination)
    en_v4_seg108: Rejected (overlap contamination)
    en_v4_seg109: Rejected (overlap contamination)
    en_v4_seg111: Rejected (overlap contamination)
    en_v4_seg114: Rejected (overlap contamination)
    en_v4_seg115: Rejected (overlap contamination)
    en_v4_seg119: Rejected (overlap contamination)
    en_v4_seg122: Rejected (overlap contamination)
    en_v4_seg123: Rejected (overlap contamination)
    en_v4_seg125: Rejected (overlap contamination)
  Turns: 101 | Accepted: 83 | Rejected: 56

Total: 83 segments | 20.6 min
```

```
from collections import defaultdict
speaker_stats = defaultdict(lambda: {"clips": 0, "duration_sec": 0.0})
for seg in all_accepted:
    spk = seg.get("speaker_id", "unknown")
    speaker_stats[spk]["clips"]        += 1
    speaker_stats[spk]["duration_sec"] += seg["duration_sec"]

print(f"{'Speaker':<25} {'Clips':>6} {'Duration':>12}")
print("-" * 46)
for spk, stats in sorted(speaker_stats.items()):
    mins = stats["duration_sec"] / 60
    flag = "  <- low yield" if mins < 1.0 else ""
    print(f"{spk:<25} {stats['clips']:>6} {mins:>10.1f} min{flag}")

print(f"\nTotal accepted : {len(all_accepted)} clips")
print(f"Total rejected : {len(all_rejected)} clips")
```

```
Speaker                    Clips     Duration
----------------------------------------------
en_v4_spk0                    15        3.8 min
en_v4_spk1                    18        4.3 min
en_v4_spk2                     7        1.6 min
en_v4_spk3                    12        3.2 min
en_v4_spk4                    31        7.8 min

Total accepted : 83 clips
Total rejected : 56 clips
```

```
import random

if all_accepted:
    pick = random.choice(all_accepted)
    print(f"ID         : {pick['id']}")
    print(f"Speaker    : {pick['speaker_id']}")
    print(f"Duration   : {pick['duration_sec']}s | SNR: {pick['snr_db']}dB")
    print(f"Transcript : {pick['transcript']}")
    ipd.display(ipd.Audio(pick["file"]))
else:
    print("No accepted segments yet.")
```

```
ID         : en_v4_seg060
Speaker    : en_v4_spk4
Duration   : 7.4s | SNR: 27.200000762939453dB
Transcript : Do you believe there are winners and losers at all in this war? How do you see the state of play at the end of a 10
```



```
from collections import Counter

reason_counts = Counter(r["reason"].split(":")[0] for r in all_rejected)
print(f"{'Rejection reason':<35} {'Count':>6}")

for reason, count in reason_counts.most_common():
    print(f"{reason:<35} {count:>6}")
```

```
Rejection reason                    Count
overlap_contamination                  30
too_short                              26
```

```
import pandas as pd

# Save accepted segments metadata
```

```
accepted_df = pd.DataFrame(all_accepted)[["id", "speaker_id", "duration_sec", "snr_db", "transcript", "source_url", "lang_code"
accepted_df.to_csv(f"{path}/english_accepted_segments.csv", index=False)

# Save rejection log
rejected_df = pd.DataFrame(all_rejected)[["id", "duration", "reason"]]
rejected_df.to_csv(f"{path}/english_rejection_log.csv", index=False)

print(f"Accepted segments saved: {len(accepted_df)} rows")
print(f"Rejection log saved: {len(rejected_df)} rows")
```

```
Accepted segments saved: 83 rows
Rejection log saved: 56 rows
```

```
def tag_emotion_video(transcript, language_code):
    prompt = f"""You are a TTS dataset curator. Analyze this English speech segment and assign tags.

Transcript: {transcript}

Respond ONLY with valid JSON, no markdown, no explanation:
{{"emotion": "<happy|sad|excited|angry|neutral|formal|whisper|frustrated>", "style": "<conversational|narrative|formal|instruct

    try:
        response = sarvam_client.chat.completions(
            model="sarvam-105b",
            messages=[{"role": "user", "content": prompt}],
            max_tokens=300,
            temperature=0.1,
            reasoning_effort=None
        )
        content = response.choices[0].message.content
        if content is None:
            print("  LLM returned None content")
            return None
        content = content.replace("```json", "").replace("```", "").strip()
        return json.loads(content)
    except json.JSONDecodeError as e:
        print(f"  JSON parse error: {e} | Raw: {content[:100]}")
        return None
    except Exception as e:
        print(f"  Sarvam LLM error: {e}")
        return None


def tag_emotion(transcript, language_code):
    result = tag_emotion_video(transcript, language_code)
    return result or {"emotion": "neutral", "style": "conversational",
                      "confidence": "low", "register": "colloquial"}
```

```
total = len(all_accepted)
for i, seg in enumerate(all_accepted):

    if not seg.get("transcript"):
        print(f"  [{i+1}/{total}] {seg['id']}... SKIP (no transcript)")
        seg.update({"emotion": "neutral", "style": "conversational",
                    "confidence": "low", "register": "colloquial"})
        continue

    print(f"  [{i+1}/{total}] {seg['id']}...", end=" ")
    tags = tag_emotion(seg["transcript"], "en-IN")
    seg.update(tags)
    print(f"{tags['emotion']} / {tags['style']} / conf:{tags['confidence']}")
    time.sleep(0.2)

emotions   = Counter(s.get("emotion",   "?") for s in all_accepted)
confidence = Counter(s.get("confidence","?") for s in all_accepted)
registers  = Counter(s.get("register",  "?") for s in all_accepted)

print(f"\nEmotion    : {dict(emotions)}")
print(f"Confidence : {dict(confidence)}")
print(f"Register   : {dict(registers)}")
```

```
[31/83] en_v4_seg039... neutral / narrative / conf:high
[32/83] en_v4_seg040... neutral / conversational / conf:high
[33/83] en_v4_seg042... neutral / narrative / conf:high
[34/83] en_v4_seg050... angry / conversational / conf:high
[35/83] en_v4_seg051... neutral / formal / conf:high
[36/83] en_v4_seg052... neutral / conversational / conf:high
[37/83] en_v4_seg053... neutral / conversational / conf:high
[38/83] en_v4_seg060... neutral / conversational / conf:high
[39/83] en_v4_seg061... frustrated / conversational / conf:high
[40/83] en_v4_seg062... frustrated / narrative / conf:high
[41/83] en_v4_seg065... neutral / conversational / conf:high
[42/83] en_v4_seg066... angry / narrative / conf:high
[43/83] en_v4_seg067... neutral / narrative / conf:high
[44/83] en_v4_seg068... neutral / conversational / conf:high
[45/83] en_v4_seg069... neutral / conversational / conf:medium
[46/83] en_v4_seg070... neutral / narrative / conf:high
[47/83] en_v4_seg071... neutral / conversational / conf:high
[48/83] en_v4_seg072... neutral / narrative / conf:high
[49/83] en_v4_seg073... neutral / conversational / conf:high
[50/83] en_v4_seg074... neutral / conversational / conf:high
[51/83] en_v4_seg075... neutral / conversational / conf:high
[52/83] en_v4_seg078... neutral / conversational / conf:high
[53/83] en_v4_seg079... neutral / conversational / conf:high
[54/83] en_v4_seg080... neutral / conversational / conf:high
[55/83] en_v4_seg081... neutral / conversational / conf:high
[56/83] en_v4_seg082... neutral / narrative / conf:high
[57/83] en_v4_seg085... neutral / conversational / conf:high
[58/83] en_v4_seg086... frustrated / conversational / conf:high
[59/83] en_v4_seg087... neutral / narrative / conf:high
[60/83] en_v4_seg088... neutral / conversational / conf:high
[61/83] en_v4_seg089... neutral / narrative / conf:high
[62/83] en_v4_seg090... neutral / conversational / conf:high
[63/83] en_v4_seg093... neutral / conversational / conf:high
[64/83] en_v4_seg094... neutral / conversational / conf:medium
[65/83] en_v4_seg096... frustrated / conversational / conf:medium
[66/83] en_v4_seg097... neutral / conversational / conf:medium
[67/83] en_v4_seg105... neutral / conversational / conf:medium
[68/83] en_v4_seg106... neutral / conversational / conf:medium
[69/83] en_v4_seg107... neutral / conversational / conf:medium
[70/83] en_v4_seg112... neutral / conversational / conf:medium
[71/83] en_v4_seg116... neutral / conversational / conf:high
[72/83] en_v4_seg121... neutral / narrative / conf:medium
[73/83] en_v4_seg126... neutral / narrative / conf:high
[74/83] en_v4_seg127... neutral / conversational / conf:high
[75/83] en_v4_seg128... neutral / conversational / conf:high
[76/83] en_v4_seg129... neutral / conversational / conf:high
[77/83] en_v4_seg130... neutral / conversational / conf:high
[78/83] en_v4_seg131... neutral / conversational / conf:high
[79/83] en_v4_seg134... neutral / conversational / conf:medium
[80/83] en_v4_seg135... neutral / conversational / conf:medium
[81/83] en_v4_seg136... neutral / narrative / conf:high
[82/83] en_v4_seg137... neutral / narrative / conf:high
[83/83] en_v4_seg138... neutral / narrative / conf:medium
```

```
SOURCE_META = {
    "en_v4": "English Panel Video 4"
}

final_metadata = []
for idx, seg in enumerate(all_accepted):
    video_id = "_".join(seg["id"].split("_")[:2])
    entry = {
      "id":              f"eng_{idx+1:04d}",
      "file_path":       seg["file"],
      "language":        "en-IN",
      "language_name":   "English",
      "speaker_id":      seg["speaker_id"],
      "duration_sec":    float(seg["duration_sec"]),
      "transcript":      seg["transcript"],
      "emotion":         seg.get("emotion",    "neutral"),
      "style":           seg.get("style",      "conversational"),
      "register":        seg.get("register",   "colloquial"),
      "tag_confidence": seg.get("confidence", "low"),
      "snr_db":          float(seg["snr_db"]),
      "source_url":      seg["source_url"],
      "source_channel": SOURCE_META.get(video_id, ""),
      "curated_at":      datetime.now().isoformat()
    }
    final_metadata.append(entry)

# Save JSONL
with open(f"{path}/metadata.jsonl", "w", encoding="utf-8") as f:
    for e in final_metadata:
```

```
        f.write(json.dumps(e, ensure_ascii=False) + "\n")

# Save rejection log
with open(f"{path}/rejected_log.jsonl", "w", encoding="utf-8") as f:
    for r in all_rejected:
        f.write(json.dumps(r, ensure_ascii=False) + "\n")

total_dur = sum(e["duration_sec"] for e in final_metadata) / 60
rej_rate  = len(all_rejected) / max(1, len(all_rejected) + len(final_metadata)) * 100

print(f"English")
print(f"  Segments  : {len(final_metadata):>4} | {total_dur:.1f} min")
print(f"  Rejected  : {len(all_rejected):>4} segments ({rej_rate:.1f}% rejection rate)")
```

```
English
  Segments  :   83 | 20.6 min
  Rejected  :   56 segments (40.3% rejection rate)
```

```
sample = random.sample(final_metadata, min(12, len(final_metadata)))
print(f"{'ID':<20} {'Dur':>5} {'SNR':>5} {'Emotion':<12} {'Conf':<8} Transcript[:70]")
print("-" * 100)
for e in sample:
    print(f"{e['id']:<20} {e['duration_sec']:>4.0f}s "
          f"{e['snr_db']:>4.0f}dB {e['emotion']:<12} {e['tag_confidence']:<8} "
          f"{e['transcript'][:70]}")

print("\nPlay a random segment")
pick = random.choice(final_metadata)
print(f"ID         : {pick['id']}")
print(f"Transcript : {pick['transcript']}")
print(f"Emotion    : {pick['emotion']} | Register: {pick['register']}")
ipd.display(ipd.Audio(pick["file_path"]))
```

```
ID                    Dur   SNR Emotion      Conf     Transcript[:70]
-----------------------------------------------------------------------------------------------
eng_0025              15s   33dB neutral      high     it's not America's first policy. America's don't have any national int
eng_0038               7s   27dB neutral      high     Do you believe there are winners and losers at all in this war? How do
eng_0006              15s   37dB neutral      high     Well, first of all, we should emphasize that the entire world benefits
eng_0052              15s   35dB neutral      high     You know, given what you've just said, Mr. Painter, you see, all of th
eng_0002              15s   26dB neutral      high     The upper hand in the US Iran deal, can Donald Trump claim victory of
eng_0016              15s   35dB neutral      high     Well, I first of all, the United States has no business calling for re
eng_0077              15s   37dB neutral      high     in the process of choosing sides, we have taken our eyes off the ball.
eng_0043              21s   40dB neutral      high     States has come out poorly because it has lost the notion that a super
eng_0048              15s   34dB neutral      high     Yes, and it is a serious development because the Strait of Hormuz is n
eng_0050              20s   33dB neutral      high     that countries like America are the first ones to break the law when i
eng_0021              15s   33dB neutral      medium   all sources about it, we can find out, for example, one of the things
eng_0083              19s   25dB neutral      medium   Iran gets a slight upper hand in the war, but Friday is quite far by T

Play a random segment
ID         : eng_0009
Transcript : to a nuclear weapons program, and I think in the long run the United States is going to insist on that, and so was
Emotion    : neutral | Register: formal
```

0:02 / 0:15

```
Start coding or generate with AI.
```